Laboratory 6
Attention mechanisms

1. Introduction

- the **attention mechanism** is an effective method for improving the performance of neural networks;
- it was proposed to enhance the NLP models, and later becomes one of the most effective methods for various machine learning tasks;
- a neural network with attention-based components can automatically distinguish the relative importance of features by means of assigning the corresponding weights to them to get a higher classification or regression accuracy;
- the main idea behind the attention mechanism stems from the human perception property to concentrate on an important part of information and to ignore other information;

2. Attention mechanism components

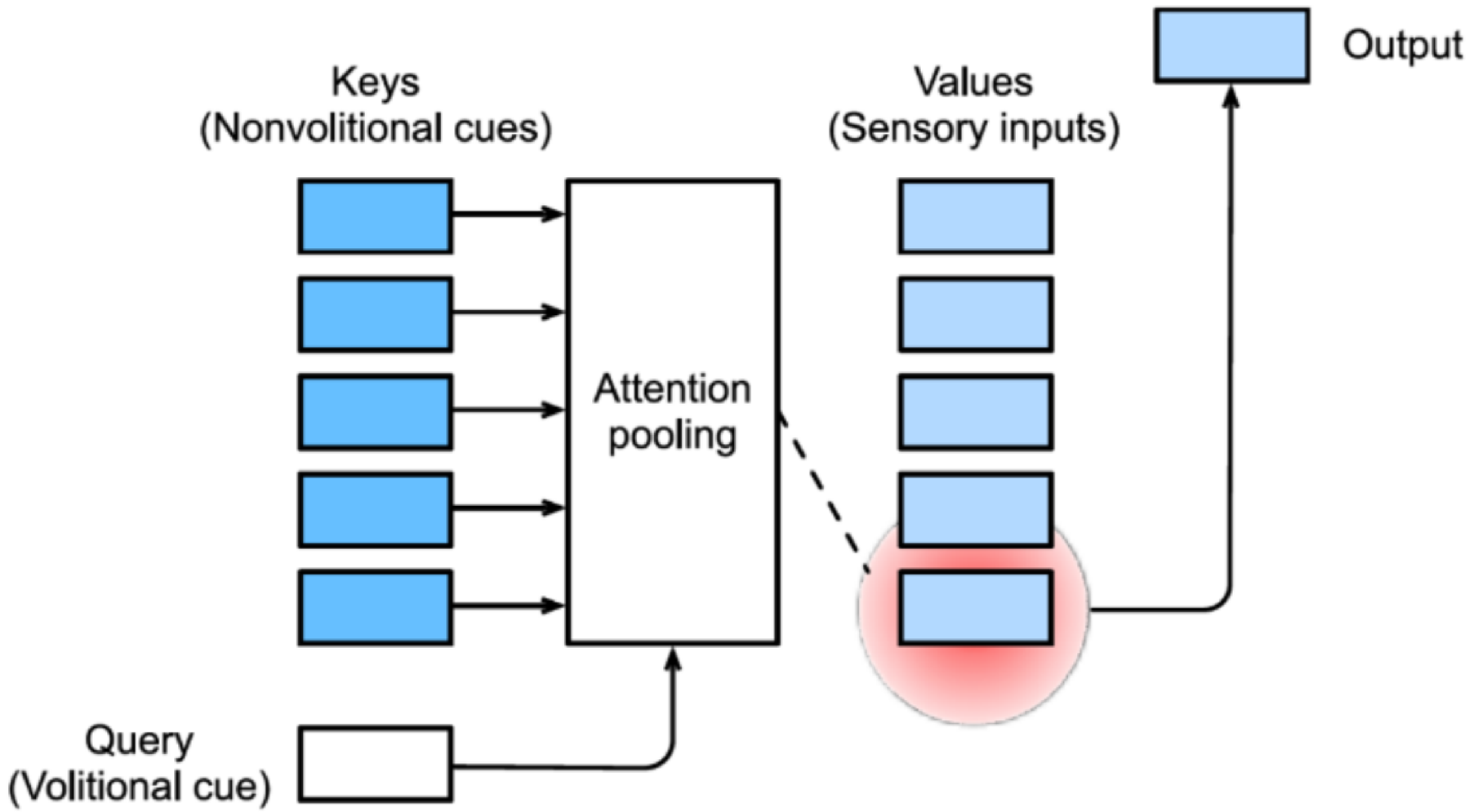


Figure 1: Attention mechanisms bias selection over values via attention pooling, which incorporates queries and keys.

3. Attention pooling: Nadaraya-Watson kernel regression

- the *Nadaraya-Watson kernel regression* model is a simple example for demonstrating machine learning with attention mechanisms;
- regression problem: given a dataset of input-output pairs $\{(x_1, y_1), \ldots, (x_n, y_n)\}$, how to learn f to predict the output $\hat{y} = f(x)$ for any new input x?
- weigh the outputs y_i according to their input locations:

$$f(x) = \sum_{i=1}^{n} \frac{K(x - x_i)}{\sum_{j=1}^{n} K(x - x_j)} y_i, \tag{1}$$

where K is a *kernel*;

3. Attention pooling: Nadaraya-Watson kernel regression

- rewrite (1) as:

$$f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i, \tag{2}$$

where x is the *query*, (x_i, y_i) is the *key-value* pair and $\alpha(x, x_i)$ is the attention weight;

- plugging the Gaussian kernel $K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right)$ into (1) gives:

$$f(x) = \sum_{i=1}^{n} \text{softmax}\left(-\frac{1}{2}(x - x_i)^2\right) y_i. \tag{3}$$

- (3) is an example of *nonparametric attention pooling*;

3. Attention pooling: Nadaraya-Watson kernel regression

- nonetheless, we can integrate learnable parameters into attention pooling;
- for example,

$$f(x) = \sum_{i=1}^{n} \text{softmax}\left(-\frac{1}{2}((x - x_i)\mathbf{w}\right)^2)y_i. \tag{4}$$

3.1 Nadaraya-Watson kernel regression in PyTorch

- nonparametric model:

```
n_train = 50  # No. of training examples
x_train, _ = torch.sort(torch.rand(n_train) * 5)   # Training inputs (keys)

def f(x):
    return 2 * torch.sin(x) + x**0.8

y_train = f(x_train) + torch.normal(0.0, 0.5, (n_train,))  # Training outputs
x_test = torch.arange(0, 5, 0.1)  # Testing examples (queries)
y_truth = f(x_test)  # Ground-truth outputs for the testing examples
n_test = len(x_test)  # No. of testing examples

X_repeat = x_test.repeat_interleave(n_train).reshape((-1, n_train))
attention_weights = F.softmax(-(X_repeat - x_train)**2 / 2, dim=1)
y_hat = torch.matmul(attention_weights, y_train)
```

3.1 Nadaraya-Watson kernel regression in PyTorch

- parametric model:

```
class NWKernelRegression(nn.Module):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.w = nn.Parameter(torch.rand((1,), requires_grad=True))

    def forward(self, queries, keys, values):
        queries = queries.repeat_interleave(keys.shape[1]).reshape((-1, keys.shape[1]))
        self.attention_weights = F.softmax(-((queries - keys) * self.w)**2 / 2, dim=1)
        return torch.bmm(self.attention_weights.unsqueeze(1),
                         values.unsqueeze(-1)).reshape(-1)

net = NWKernelRegression()
loss = nn.MSELoss(reduction='none')
optimizer = torch.optim.SGD(net.parameters(), lr=0.5)

train_loss_all = []
for epoch in range(5):
    optimizer.zero_grad()
    l = loss(net(x_train, keys, values), y_train)
    l.sum().backward()
    optimizer.step()
    print(f'epoch {epoch + 1}, loss {float(l.sum()):.6f}')
    train_loss_all.append(float(l.sum()))
```

4. Attention scoring functions

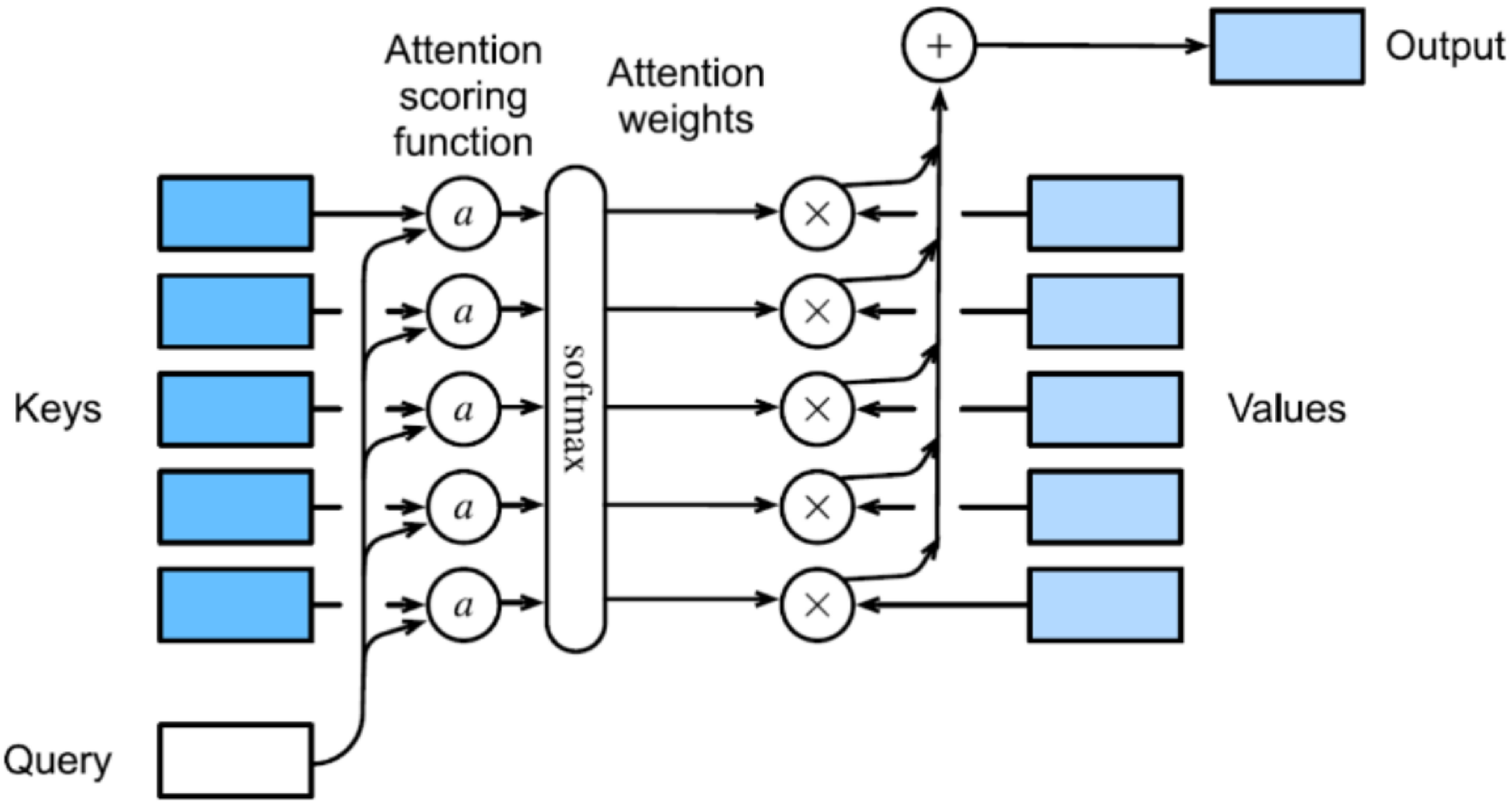


Figure 2: Computing the output of attention pooling as a weighted average of values (a – attention scoring function).

4. Attention scoring functions

- suppose we have a query $\mathbf{q} \in \mathbb{R}^q$ and m key-value pairs $(\mathbf{k_1}, \mathbf{v_1}), \ldots, (\mathbf{k_m}, \mathbf{v_m})$, where $\mathbf{k_i} \in \mathbb{R}^k$ and $\mathbf{v_i} \in \mathbb{R}^v$;
- the attention pooling f is instantiated as a weighted sum of the values:

$$f(\mathbf{q}, (\mathbf{k_1}, \mathbf{v_1}), \ldots, (\mathbf{k_m}, \mathbf{v_m})) = \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k_i}) \mathbf{v_i} \in \mathbb{R}^v, \tag{5}$$

where the attention weight (scalar) for the query $\mathbf{q}$ and key $\mathbf{k_i}$ is computed by the softmax operation of an attention scoring function a that maps two vectors to a scalar:

$$\alpha(\mathbf{q}, \mathbf{k_i}) = softmax(a(\mathbf{q}, \mathbf{k_i})) = \frac{exp(a(\mathbf{q}, \mathbf{k_i}))}{\sum_{j=1}^{m} exp(a(\mathbf{q}, \mathbf{k_i}))} \in \mathbb{R}. \tag{6}$$

4. Attention scoring functions

- scoring function $\rightarrow$ <u>additive attention</u> defined by:

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w_v}^T tanh(\mathbf{W_q q} + \mathbf{W_k k}) \in \mathbb{R}, \tag{7}$$

where the parameters $\mathbf{W_q} \in \mathbb{R}^{h \times q}$, $\mathbf{W_k} \in \mathbb{R}^{h \times k}$, $\mathbf{w_v} \in \mathbb{R}^h$ are learnable, and $\mathbf{q} \in \mathbb{R}^q$, $\mathbf{k} \in \mathbb{R}^k$ are query and key.

- scoring function $\rightarrow$ <u>scaled dot-product attention</u> defined by:

$$a(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}^T \mathbf{k}}{\sqrt{d}}, \tag{8}$$

where d is the length of query and key vectors.

4.1 Attention scoring functions in PyTorch

- additive attention:

```
class AdditiveAttention(nn.Module):
    """Additive attention."""
    def __init__(self, key_size, query_size, num_hiddens, dropout, **kwargs):
        super(AdditiveAttention, self).__init__(**kwargs)
        self.W_k = nn.Linear(key_size, num_hiddens, bias=False)
        self.W_q = nn.Linear(query_size, num_hiddens, bias=False)
        self.w_v = nn.Linear(num_hiddens, 1, bias=False)
        self.dropout = nn.Dropout(dropout)

    def forward(self, queries, keys, values, valid_lens):
        queries, keys = self.W_q(queries), self.W_k(keys)

        features = queries.unsqueeze(2) + keys.unsqueeze(1)
        features = torch.tanh(features)

        scores = self.w_v(features).squeeze(-1)
        self.attention_weights = masked_softmax(scores, valid_lens)

        return torch.bmm(self.dropout(self.attention_weights), values)
```

4.1 Attention scoring functions in PyTorch

- scaled dot-product attention:

```
class DotProductAttention(nn.Module):
    """Scaled dot product attention."""
    def __init__(self, dropout, **kwargs):
        super(DotProductAttention, self).__init__(**kwargs)
        self.dropout = nn.Dropout(dropout)

    def forward(self, queries, keys, values, valid_lens=None):
        d = queries.shape[-1]
        scores = torch.bmm(queries, keys.transpose(1,2)) / math.sqrt(d)
        self.attention_weights = masked_softmax(scores, valid_lens)
        return torch.bmm(self.dropout(self.attention_weights), values)
```

5. Bahdanau attention

- this attention mechanism is employed in the RNN encoder-decoder architecture studied in the context of machine translation problem;
- the resulting attention model replaces the context variable $\mathbf{c}$ (the output of the encoder) with $\mathbf{c_{t'}}$ at any decoding time step t';
- suppose there are T tokens in the input sequence, then the context variable at the decoding time step t' is the output of attention pooling:

$$\mathbf{c_{t'}} = \sum_{t=1}^{T} \alpha(\mathbf{s_{t'-1}}, \mathbf{h_t})\mathbf{h_t}, \tag{9}$$

where the decoder hidden state $\mathbf{s_{t'-1}}$ at time step $t' - 1$ is the query, and the encoder hidden states $\mathbf{h_t}$ are both the keys and values, and the attention weight α is computed as in (6), using the additive attention scoring function defined by (7).

5.1 Bahdanau attention in PyTorch

```
class Seq2SeqAttentionDecoder(AttentionDecoder):
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                 dropout=0, **kwargs):
        super(Seq2SeqAttentionDecoder, self).__init__(**kwargs)
        self.attention = AdditiveAttention(
            num_hiddens, num_hiddens, num_hiddens, dropout)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(
            embed_size + num_hiddens, num_hiddens, num_layers,
            dropout=dropout)
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, enc_valid_lens, *args):
        outputs, hidden_state = enc_outputs
        return (outputs.permute(1, 0, 2), hidden_state, enc_valid_lens)

    def forward(self, X, state):
        enc_outputs, hidden_state, enc_valid_lens = state
        X = self.embedding(X).permute(1, 0, 2)
        outputs, self._attention_weights = [], []
        for x in X:
            query = torch.unsqueeze(hidden_state[-1], dim=1)
            context = self.attention(
                query, enc_outputs, enc_outputs, enc_valid_lens)
            x = torch.cat((context, torch.unsqueeze(x, dim=1)), dim=-1)
            out, hidden_state = self.rnn(x.permute(1, 0, 2), hidden_state)
            outputs.append(out)
            self._attention_weights.append(self.attention.attention_weights)
        outputs = self.dense(torch.cat(outputs, dim=0))
        return outputs.permute(1, 0, 2), [enc_outputs, hidden_state,
                                          enc_valid_lens]
```

6. Multi-head attention

- h projected queries, keys, and values are fed into attention pooling in parallel;
- in the end, the h attention pooling outputs are concatenated and transformed with another learned linear projection, to produce the final output;
- this design is called **multi-head attention**, where each of the h attention pooling outputs is a *head*;
- using fully-connected layers to perform learnable linear transformations, Figure 3 describes multi-head attention;

6. Multi-head attention

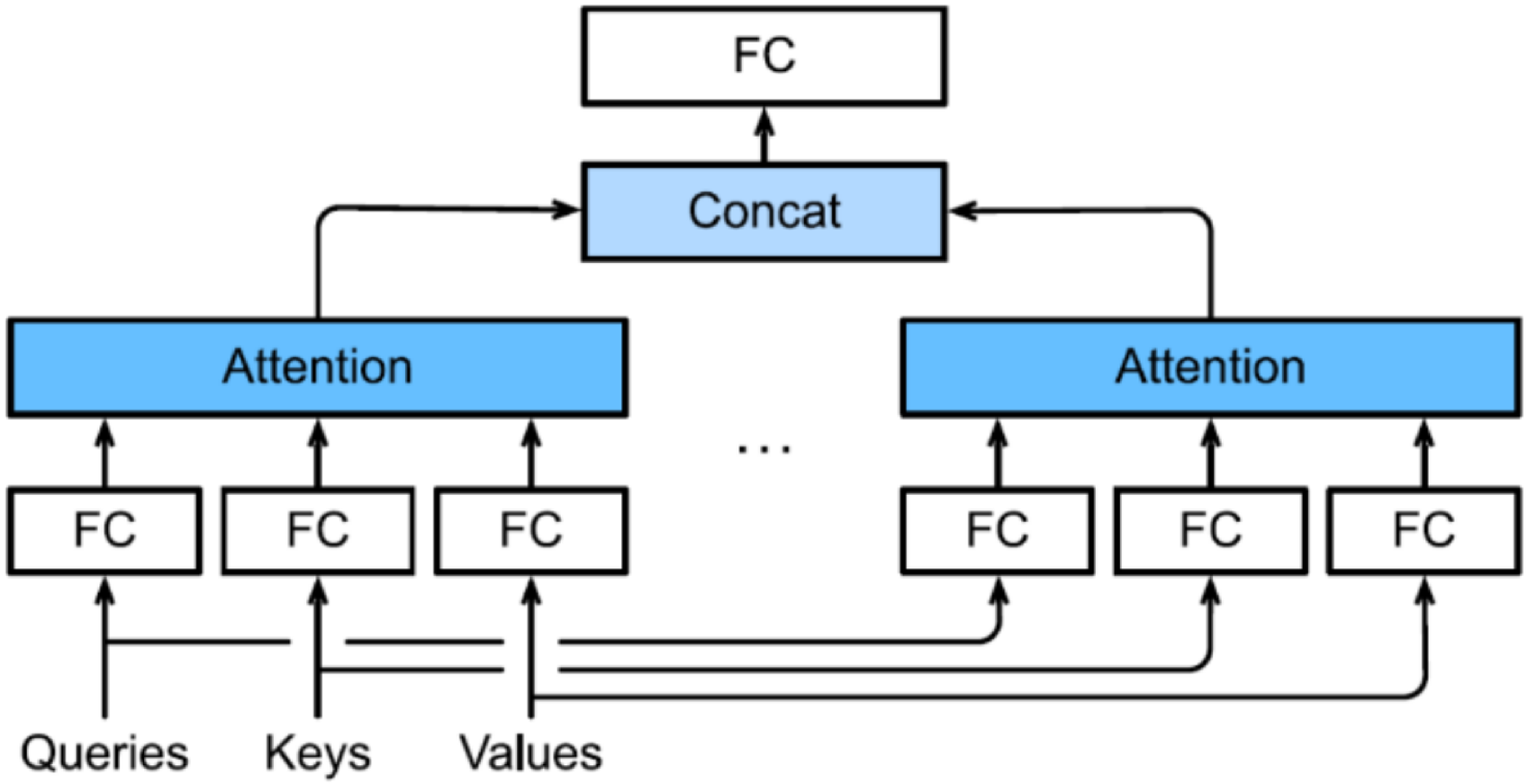


Figure 3: Multi-head attention, where multiple heads are concatenated, then linearly transformed.

6. Multi-head attention

- mathematically, given a query $\mathbf{q} \in \mathbb{R}^{d_q}$, a key $\mathbf{k} \in \mathbb{R}^{d_k}$, and a value $\mathbf{v} \in \mathbb{R}^{d_v}$, each attention head $\mathbf{h_i}(i = 1, \ldots, h)$ is computed as:

$$\mathbf{h_i} = f(\mathbf{W_i^{(q)}}\mathbf{q}, \mathbf{W_i^{(k)}}\mathbf{k}, \mathbf{W_i^{(v)}}\mathbf{v}) \in \mathbb{R}^{p_v},$$

where the parameters $\mathbf{W_i^{(q)}} \in \mathbb{R}^{p_q \times d_q}$, $\mathbf{W_i^{(k)}} \in \mathbb{R}^{p_k \times d_k}$, and $\mathbf{W_i^{(v)}} \in \mathbb{R}^{p_v \times d_v}$ are learnable, and f is attention pooling, such as additive attention or scaled dot-product attention;

- the multi-head attention output is another linear transformation via learnable parameters $\mathbf{W_o} \in \mathbb{R}^{p_o \times hp_v}$ of the concatenation of the h heads:

$$\mathbf{W_o} \begin{bmatrix} \mathbf{h_1} \\ \vdots \\ \mathbf{h_h} \end{bmatrix} \in \mathbb{R}^{p_o}.$$

6.1 Multi-head attention in PyTorch

- p_o is specified via the argument *num_hiddens*;

```
class MultiHeadAttention(nn.Module):
    """Multi-head attention."""
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                 num_heads, dropout, bias=False, **kwargs):
        super(MultiHeadAttention, self).__init__(**kwargs)
        self.num_heads = num_heads
        self.attention = DotProductAttention(dropout)
        self.W_q = nn.Linear(query_size, num_hiddens, bias=bias)
        self.W_k = nn.Linear(key_size, num_hiddens, bias=bias)
        self.W_v = nn.Linear(value_size, num_hiddens, bias=bias)
        self.W_o = nn.Linear(num_hiddens, num_hiddens, bias=bias)
```

```
def forward(self, queries, keys, values, valid_lens):
    # Shape of `queries`, `keys`, or `values` is:
    # (`batch_size`, no. of queries or key-value pairs, `num_hiddens`)
    # Shape of `valid_lens` is:
    # (`batch_size`,) or (`batch_size`, no. of queries)
    # After transposing, shape of output `queries`, `keys`, or `values` is:
    # (`batch_size` * `num_heads`, no. of queries or key-value pairs,
    # `num_hiddens` / `num_heads`)
    queries = transpose_qkv(self.W_q(queries), self.num_heads)
    keys = transpose_qkv(self.W_k(keys), self.num_heads)
    values = transpose_qkv(self.W_v(values), self.num_heads)

    if valid_lens is not None:
        valid_lens = torch.repeat_interleave(
            valid_lens, repeats=self.num_heads, dim=0)

    # Shape of `output` is: (`batch_size` * `num_heads`, no. of queries,
    # `num_hiddens` / `num_heads`)
    output = self.attention(queries, keys, values, valid_lens)

    # Shape of `output_concat` is:
    # (`batch_size`, no. of queries, `num_hiddens`)
    output_concat = transpose_output(output, self.num_heads)
    return self.W_o(output_concat)
```

7. Self-attention and positional encoding

- sequence encoding using self-attention and additional information for the sequence order;
- **self-attention** (**intra-attention**) $\rightarrow$ queries, keys and values are from the same source;
- given a sequence of input tokens $\mathbf{x_1}, \ldots, \mathbf{x_n}$, where any $\mathbf{x_i} \in \mathbb{R}^d$ $(1 \leq i \leq n)$, its self-attention outputs a sequence of the same length $\mathbf{y_1}, \ldots, \mathbf{y_n}$, where:

$$\mathbf{y_i} = f(\mathbf{x_i}, (\mathbf{x_1}, \mathbf{x_1}), \ldots, (\mathbf{x_n}, \mathbf{x_n})) \in \mathbb{R}^d,$$

according to the definition of attention pooling f in (5).

7. Self-attention and positional encoding

- to use the sequence order information, we can inject absolute or relative positional information by adding **positional encoding** to the input representations;
- positional encodings can be either learned or fixed; in the following, we describe a fixed positional encoding based on sine and cosine functions;
- suppose that the input representation $\mathbf{X} \in \mathbb{R}^{n \times d}$ contains $d-$dimensional embeddings for n tokens of a sequence;

7. Self-attention and positional encoding

- the positional encoding outputs $\mathbf{X} + \mathbf{P}$ using a positional embedding matrix $\mathbf{P} \in \mathbb{R}^{n \times d}$ of the same shape, whose element on the ith row and the $(2j)$th or the $(2j+1)$th column is:

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right).$$

- in the positional embedding matrix $\mathbf{P}$, rows correspond to positions within a sequence and columns represent different positional encoding dimensions;

7.1 Self-attention and positional encoding in PyTorch

- positional encoding based on sine and cosine functions;

```
class PositionalEncoding(nn.Module):
    """Positional encoding."""
    def __init__(self, num_hiddens, dropout, max_len=1000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(dropout)
        # Create a long enough `P`
        self.P = torch.zeros((1, max_len, num_hiddens))
        X = torch.arange(max_len, dtype=torch.float32).reshape(
            -1, 1) / torch.pow(10000, torch.arange(
            0, num_hiddens, 2, dtype=torch.float32) / num_hiddens)
        self.P[:, :, 0::2] = torch.sin(X)
        self.P[:, :, 1::2] = torch.cos(X)

    def forward(self, X):
        X = X + self.P[:, :X.shape[1], :].to(X.device)
        return self.dropout(X)
```

8. Transformer model

- the **Transformer** model is solely based on attention mechanisms, without any convolutional or recurrent layer;
- the Transformer is composed of an encoder and a decoder, Figure 4;
- the input and output sequence embeddings are added with positional encoding, before being fed into the encoder and the decoder that stack modules based on self-attention;

8. Transformer model

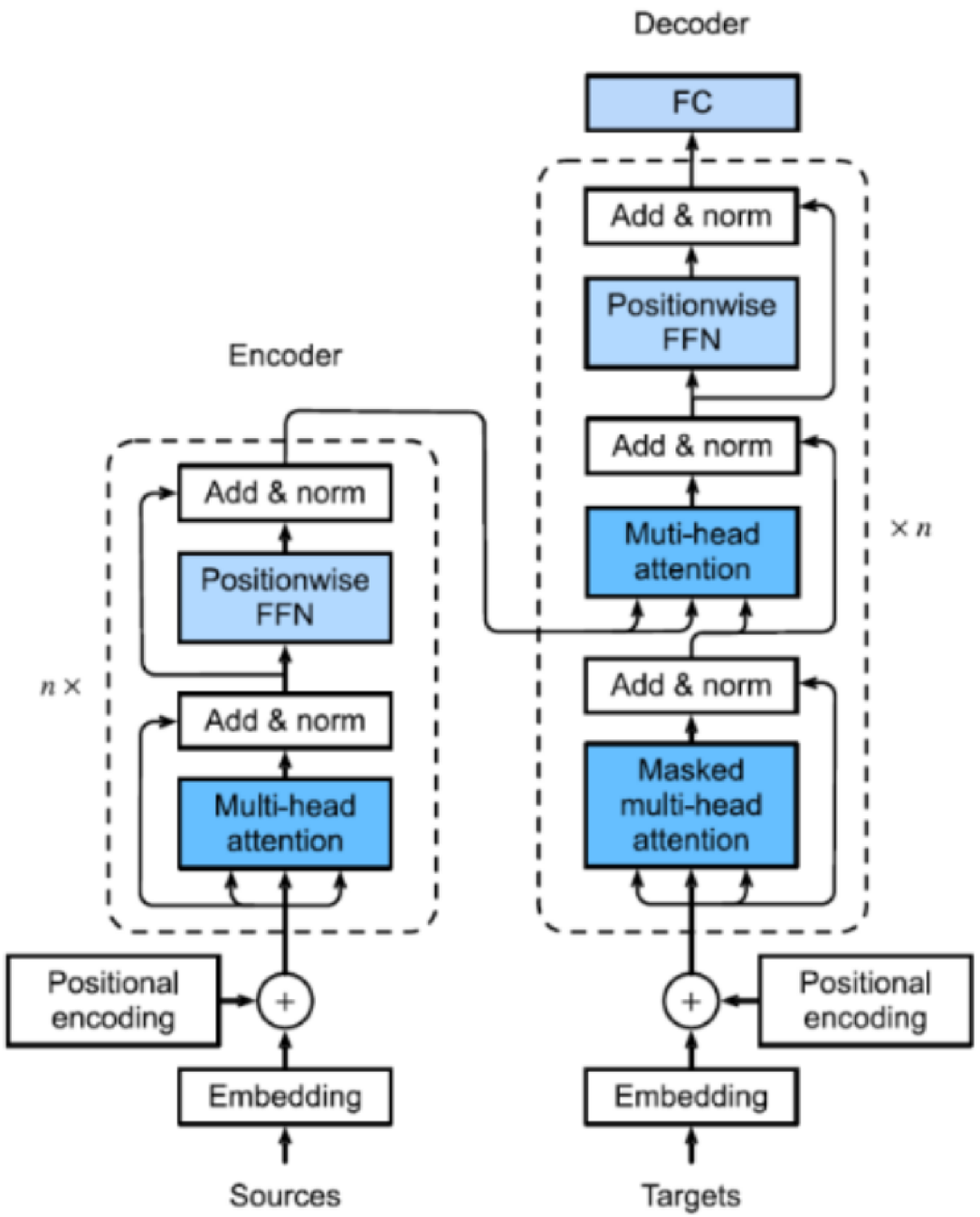


Figure 4: The Transformer architecture.

8. Transformer model

- the Transformer encoder is a stack of multiple identical layers, where each layer has two sublayers;
- the first is a multi-head self-attention pooling and the second is a position-wise feed-forward network;
- specifically, in the encoder self-attention, queries, keys, and values are all from the outputs of the previous encoder layer;
- inspired by the ResNet design, a residual connection is employed around both sublayers;

8. Transformer model

- the Transformer decoder is also a stack of multiple identical layers with residual connections and layer normalizations;
- besides the two sublayers described in the encoder, the decoder inserts a third sublayer, known as the *encoder-decoder attention*, between these two;
- in the encoder-decoder attention, queries are from the outputs of the previous decoder layer, and the keys and values are from the Transformer encoder outputs.

8.1 Transformer model in PyTorch

- Positionwise FFN sublayer implementation;

```
class PositionWiseFFN(nn.Module):
    """Position-wise feed-forward network."""
    def __init__(self, ffn_num_input, ffn_num_hiddens, ffn_num_outputs,
                 **kwargs):
        super(PositionWiseFFN, self).__init__(**kwargs)
        self.dense1 = nn.Linear(ffn_num_input, ffn_num_hiddens)
        self.relu = nn.ReLU()
        self.dense2 = nn.Linear(ffn_num_hiddens, ffn_num_outputs)

    def forward(self, X):
        return self.dense2(self.relu(self.dense1(X)))
```

- Add & norm implementation;

```
class AddNorm(nn.Module):
    """Residual connection followed by layer normalization."""
    def __init__(self, normalized_shape, dropout, **kwargs):
        super(AddNorm, self).__init__(**kwargs)
        self.dropout = nn.Dropout(dropout)
        self.ln = nn.LayerNorm(normalized_shape)

    def forward(self, X, Y):
        return self.ln(self.dropout(Y) + X)
```

8.1 Transformer model in PyTorch

- *EncoderBlock* class contains two sublayers: multi-head self-attention and position-wise feed-forward networks;

```
class EncoderBlock(nn.Module):
    """Transformer encoder block."""
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                 norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
                 dropout, use_bias=False, **kwargs):
        super(EncoderBlock, self).__init__(**kwargs)
        self.attention = MultiHeadAttention(
            key_size, query_size, value_size, num_hiddens, num_heads, dropout,
            use_bias)
        self.addnorm1 = AddNorm(norm_shape, dropout)
        self.ffn = PositionWiseFFN(
            ffn_num_input, ffn_num_hiddens, num_hiddens)
        self.addnorm2 = AddNorm(norm_shape, dropout)

    def forward(self, X, valid_lens):
        Y = self.addnorm1(X, self.attention(X, X, X, valid_lens))
        return self.addnorm2(Y, self.ffn(Y))
```

8.1 Transformer model in PyTorch

- *TransformerEncoder* → *num_layers* instances of the *EncoderBlock* class are stacked;

```
class TransformerEncoder(Encoder):
    """Transformer encoder."""
    def __init__(self, vocab_size, key_size, query_size, value_size,
                 num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens,
                 num_heads, num_layers, dropout, use_bias=False, **kwargs):
        super(TransformerEncoder, self).__init__(**kwargs)
        self.num_hiddens = num_hiddens
        self.embedding = nn.Embedding(vocab_size, num_hiddens)
        self.pos_encoding = PositionalEncoding(num_hiddens, dropout)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            self.blks.add_module("block"+str(i),
                EncoderBlock(key_size, query_size, value_size, num_hiddens,
                             norm_shape, ffn_num_input, ffn_num_hiddens,
                             num_heads, dropout, use_bias))

    def forward(self, X, valid_lens, *args):
        # Since positional encoding values are between -1 and 1, the embedding
        # values are multiplied by the square root of the embedding dimension
        # to rescale before they are summed up
        X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
        self.attention_weights = [None] * len(self.blks)
        for i, blk in enumerate(self.blks):
            X = blk(X, valid_lens)
            self.attention_weights[
                i] = blk.attention.attention.attention_weights
        return X
```

8.1 Transformer model in PyTorch

- decoder block implementation;

```
class DecoderBlock(nn.Module):
    # The `i`-th block in the decoder
    def __init__(self, key_size, query_size, value_size, num_hiddens,
                 norm_shape, ffn_num_input, ffn_num_hiddens, num_heads,
                 dropout, i, **kwargs):
        super(DecoderBlock, self).__init__(**kwargs)
        self.i = i
        self.attention1 = MultiHeadAttention(
            key_size, query_size, value_size, num_hiddens, num_heads, dropout)
        self.addnorm1 = AddNorm(norm_shape, dropout)
        self.attention2 = MultiHeadAttention(
            key_size, query_size, value_size, num_hiddens, num_heads, dropout)
        self.addnorm2 = AddNorm(norm_shape, dropout)
        self.ffn = PositionWiseFFN(ffn_num_input, ffn_num_hiddens,
                                   num_hiddens)
        self.addnorm3 = AddNorm(norm_shape, dropout)

    def forward(self, X, state):
        enc_outputs, enc_valid_lens = state[0], state[1]
        if state[2][self.i] is None:
            key_values = X
        else:
            key_values = torch.cat((state[2][self.i], X), axis=1)
        state[2][self.i] = key_values
        if self.training:
            batch_size, num_steps, _ = X.shape
            dec_valid_lens = torch.arange(
                1, num_steps + 1, device=X.device).repeat(batch_size, 1)
        else:
            dec_valid_lens = None
        # Self-attention
        X2 = self.attention1(X, key_values, key_values, dec_valid_lens)
        Y = self.addnorm1(X, X2)
        Y2 = self.attention2(Y, enc_outputs, enc_outputs, enc_valid_lens)
        Z = self.addnorm2(Y, Y2)
        return self.addnorm3(Z, self.ffn(Z)), state
```

8.1 Transformer model in PyTorch

- Transformer decoder composed of *num_layers* instances of *DecoderBlock*; a fully-connected layer computes the prediction for all the *vocab_size* possible output tokens.

```
class TransformerDecoder(AttentionDecoder):
    def __init__(self, vocab_size, key_size, query_size, value_size,
                 num_hiddens, norm_shape, ffn_num_input, ffn_num_hiddens,
                 num_heads, num_layers, dropout, **kwargs):
        super(TransformerDecoder, self).__init__(**kwargs)
        self.num_hiddens = num_hiddens
        self.num_layers = num_layers
        self.embedding = nn.Embedding(vocab_size, num_hiddens)
        self.pos_encoding = PositionalEncoding(num_hiddens, dropout)
        self.blks = nn.Sequential()
        for i in range(num_layers):
            self.blks.add_module("block"+str(i),
                DecoderBlock(key_size, query_size, value_size, num_hiddens,
                             norm_shape, ffn_num_input, ffn_num_hiddens,
                             num_heads, dropout, i))
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, enc_valid_lens, *args):
        return [enc_outputs, enc_valid_lens, [None] * self.num_layers]

    def forward(self, X, state):
        X = self.pos_encoding(self.embedding(X) * math.sqrt(self.num_hiddens))
        self._attention_weights = [[None] * len(self.blks) for _ in range (2)]
        for i, blk in enumerate(self.blks):
            X, state = blk(X, state)
            # Decoder self-attention weights
            self._attention_weights[0][
                i] = blk.attention1.attention.attention_weights
            # Encoder-decoder attention weights
            self._attention_weights[1][
                i] = blk.attention2.attention.attention_weights
        return self.dense(X), state
```